CSE 310 Online (A2): String and Numeric suffixes

Time: 40 minutes **Full Marks: 10**

I am borrowing from Python again. You will extend the lexical analyzer developed for the offline lexical-analysis assignment. Add the following two features to your existing `.l` file. All previously specified tokenization, line-counting, and error-reporting rules will remain unchanged.

1. Python-Style Multiline Strings — 5 marks

In addition to the existing C-style string and character constants, recognize Python-style multiline strings enclosed by either three single quotes or three double quotes:

```
'''This string may
continue on another line.'''

"""This is another
multiline string."""
```

The opening and closing delimiters must be of the **same type**. A string beginning with `"'` must end with `"'`, while one beginning with `"""` must end with `"""`. The first matching closing delimiter terminates the string.

For every valid multiline string, write one token using the existing convention:

```
<MULTILINE_STRING, actual string content>
```

The three opening and three closing quote characters are not part of the token value. Newline characters occurring inside the multiline string are retained. Escape sequences already supported by the offline assignment, such as `\n`, `\t`, and `\\`, must be converted to their actual characters.

The corresponding log entry must be:

```
Line no <start>: Token <MULTILINE_STRING> ending at line <end> found
```

Python-style multiline strings will not be inserted into the scope table.

Required error checking

If the opening and closing delimiters are not same, then report the error:

```
Line no <start>: Multiline string: Delimiter mistach
```

The error must be included in the existing total error count.

2. C-Style Numeric Suffixes — 5 marks

Extend numeric-constant recognition to support the following suffixes. The suffix is part of the complete lexeme and must also appear in the generated token.

Integer suffixes

A decimal integer may end with:

- exactly one `U` or `u`, producing `CONST_UINT` (unsigned int);
- exactly one `L` or `l`, producing `CONST_LONG_INT` with integers and `CONST_LONG_FLOAT` with floating point numbers;
- one unsigned suffix and one long suffix, in either order and in any letter case, producing `CONST_ULONG`.

Examples:

```
25U     -> <CONST_UINT, 25U>
90l     -> <CONST_LONG_INT, 90l>
120UL   -> <CONST_ULONG, 120UL>
75lu    -> <CONST_ULONG, 75lu>
9.2l     -> <CONST_LONG_FLOAT, 9.2l>
```

Floating-point suffixes

A decimal integer may end with:

- `F` or `f`, producing `CONST_FLOAT`;

```
1.25E-3F  -> <CONST_FLOAT, 1.25E-3F>
```

Required error checking

Report the following message for an invalid or repeated suffix:

```
Line no <n>: Invalid numeric suffix in <lexeme>
```

Examples of invalid lexemes include:

```
12UU     40ULU
1.2E3U   3.4UL
```

Here, there are repeated suffixes or floating point numbers ending with U/UL.

Sample I/O

For the samples below, only the additional entries produced by the new features are shown. Existing tokens, ordinary log entries, and symbol-table printouts are omitted.

Sample 1: Character constants, multiline strings, and valid suffixes

Input

```
char grade = 'A';
char quote = '\'';
char message[] = '''First line
Second line''';
int permission = 25U;
long population = 90l;
float rate = 1.25E-3F;
double distance = 9.2L;
```

The lexemes `'A'` and `'\"` must continue to produce the existing `CONST_CHAR` token. They must not be confused with the triple-single-quote delimiter.

Additional token-file content

```
<MULTILINE_STRING, First line
Second line>
<CONST_UINT, 25U>
<CONST_LONG_INT, 90l>
<CONST_SCI_FLOAT, 1.25E-3F>
<CONST_LONG_FLOAT, 9.2L>
```

Additional log-file content

```
Line no 3: Token <MULTILINE_STRING> ending at line 4 found
Line no 5: Token <CONST_UINT> Lexeme 25U found
```

```
Line no 6: Token <CONST_LONG_INT> Lexeme 90l found
Line no 7: Token <CONST_SCI_FLOAT> Lexeme 1.25E-3F found
Line no 8: Token <CONST_LONG_FLOAT> Lexeme 9.2L found
```

Sample 2: Double-quoted multiline string and unsigned-long suffixes

Input

```
char description[] = """The sequence ''' inside this string
does not close the string.""";
unsigned long first = 120UL;
unsigned long second = 75lu;
float measurement = 6.02E23f;
```

The triple-single-quote sequence inside the string is ordinary content because the string was opened using three double quotes.

Additional token-file content

```
<MULTILINE_STRING, The sequence ''' inside this string
does not close the string.>
<CONST_ULONG, 120UL>
<CONST_ULONG, 75lu>
<CONST_FLOAT, 6.02E23f>
```

Additional log-file content

```
Line no 1: Token <MULTILINE_STRING> ending at line 2 found
Line no 3: Token <CONST_ULONG> Lexeme 120UL found
Line no 4: Token <CONST_ULONG> Lexeme 75lu found
Line no 5: Token <CONST_FLOAT> Lexeme 6.02E23f found
```

Sample 3: Invalid suffixes and unfinished multiline string

Input

```
int first = 12UU;
float second = 1.2E3U;
float third = 3.4UL;
int fourth = 25F;
char unfinished[] = '''This multiline string
has delimiter mismatch"""
```

An invalid numeric lexeme must be consumed as one unit. It must not be split into a valid numeric token followed by one or more identifiers.

Additional token-file content

```
[no additional token is produced]
```

Additional log-file content

```
Line no 1: Invalid numeric suffix in 12UU
Line no 2: Invalid numeric suffix in 1.2E3U
Line no 3: Invalid numeric suffix in 3.4UL
Line no 5: Multiline string: Delimiter mismatch
```

Marks Distribution

- Recognition of strings: **4 marks**

- Unfinished multiline-string detection and correct line counting: **1 marks**
- Correct recognition and token generation for valid integer and scientific suffixes: **3 marks**
- Invalid suffix/exponent detection without splitting the lexeme: **2 marks**

Submission: Submit only your `2205ABC.l` file. Do not submit `lex.yy.c`, executables, token files, or log files.